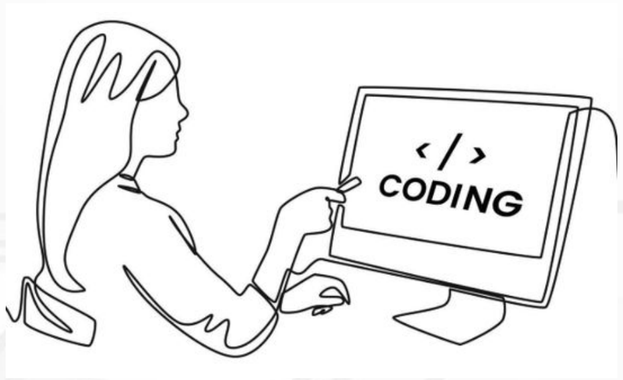
FADENA
FACULTAD DE DEFENSA NACIONAL

# Lenguajes de Programación

## Modularización - avanzada

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



# Objetivos

- Entender cómo organizar mejor un programa en C utilizando estructuras, archivos y capas de software.
- Conocer typedef y struct para definir nuevos tipos de datos.
- Aprender a dividir el código en archivos .h y .c.
- La directiva #include
- Entender la idea de arquitectura en capas en software.
- Ver un ejemplo práctico de un sistema completo en Visual Studio.



# Repaso

**¿Por qué modularizar?**

- Problema: un programa lineal crece y se vuelve inmanejable.
- Solución: divide y vencerás → separar en subprogramas.
- En C: todo son funciones (aunque algunas funcionan como procedimientos).
- Un solo bloque → difícil de mantener
- Dividir en funciones → claro, reutilizable, escalable

```c
#include <stdio.h>

// Prototipos
int sumar(int a, int b);
int restar(int a, int b);

int main() {
    int x = 5, y = 3;
    int resultado;

    resultado = sumar(x, y);
    mostrarResultado(resultado);

    resultado = restar(x, y);
    mostrarResultado(resultado);

    duplicarPorReferencia(&x);
    mostrarResultado(x);

    return 0;
}

// Definiciones
int sumar(int a, int b) {
    // return a + b;
}

int restar(int a, int b) {
    // return a - b;
}
```

# Alias en tipos de datos typedef

- **typedef** permite darle un alias a un tipo existente → más claro, más legible, más portable.
- Facilita la lectura del código.
- Evita repetir tipos largos.
- Permite adaptar fácilmente el programa si cambia el microcontrolador o compilador.
- Sintaxis general

  **typedef** tipo_existente nuevo_nombre;

```c
// Tipos de medidas y unidades
typedef float temperatura_t;
typedef double distancia_t;    // en metros
typedef float peso_t;          // en kilogramos
typedef long tiempo_t;         // en segundos

int main() {
    temperatura_t temp = 20.1;
    distancia_t distancia = 10.2;
    peso_t peso = 10;

    printf("Temperatura: %.1f°C\n", temp_actual);
    printf("Distancia de A a B: %.1f°\n", distancia);
    printf("Peso: %.1f°\n", peso);

    return 0;
}
```

# Agrupar datos relacionados: struct

- **struct** permite empaquetar variables distintas en un solo bloque lógico.
- Representa un objeto del mundo real (ej: sensor, alumno, producto).
- Se puede combinar con **typedef** para simplificar el uso

```c
struct Sensor {
    int id;
    float temperatura;
};

int main(){
    struct Sensor s1;
    s1.id = 1;
    s1.temperatura = 25.0;

    return 0;
}
```

```c
// Ejemplo
typedef struct {
    int id;
    float temperatura;
} Sensor;

int main (){
    Sensor s1 = {1, 25.0};
    ...
    return 0;
}
```

```c
typedef float temperatura_t;
typedef float humedad_t;
typedef int calidad_aire_t;
typedef long timestamp_t;

typedef struct {
    temperatura_t temp;
    humedad_t humedad;
    calidad_aire_t calidad_aire;
    timestamp_t timestamp;
} lectura_sensor_t;

// Procdimiento que usa los tipos semánticos
void procesar_lectura(lectura_sensor_t lectura) {
    printf("Lectura del sensor:\n");
    printf(" - Temperatura: %.1f°C\n", lectura.temp);
    printf(" - Humedad: %.1f%%\n", lectura.humedad);
    printf(" - Calidad aire: %d\n", lectura.calidad_aire);
    printf(" - Timestamp: %ld\n", lectura.timestamp);
}
```

# Archivos .h y .c

**Concepto**

- *Separar declaraciones y definiciones en archivos distintos hace el código más ordenado, reutilizable y escalable.*

**¿Cómo se organiza?**

- `.h` **(header file)**
  - `#define`
  - `typedef`
  - `struct`
  - prototipos de funciones

- `.c` **(source file)**
  - `#include` del `.h` correspondiente
  - implementación de las funciones

```h
// ---> sensor.h <---

#ifndef SENSOR_H
#define SENSOR_H

typedef struct {
    int id;
    float temperatura;
} Sensor;

// Prototipo de función
void leerSensor(Sensor *s);

#endif
```

```c
// ---> sensor.c <---

#include "sensor.h"
#include <stdio.h>

void leerSensor(Sensor *s) {
    s->temperatura = 25.0;   // Simulación de lectura
}
```

# Modularización: archivos .h y .c con #include

**Qué hace `#include`?**

- Inserta el contenido de un archivo dentro del código fuente **antes de compilar**.
- Permite **usar módulos externos** sin reescribir código.

**Dos formas de uso**:

**#include** <archivo> → busca en las librerías del sistema.

Ejemplo: **#include** <stdio.h>

**#include** "archivo.h" → busca en el proyecto del usuario.

Ejemplo: **#include** "sensor.h"

```c
#include <stdio.h>
#include "sensor.h"   // Incluimos el módulo

int main() {
    Sensor s1 = {1, 0};
    leerSensor(&s1);   //pasamos la direccion de s1
    printf("Sensor %d -> %.1f °C\n", s1.id, s1.temperatura);
    return 0;
}
```

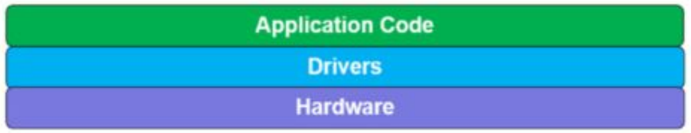
# Estructura en Capas en Programación

**Contenido**:

- La programación por capas organiza el código en diferentes niveles con responsabilidades bien definidas.
- Permite modularidad, reutilización y mantenimiento sencillo.
- Cada capa se comunica solo con la capa inmediatamente inferior o superior.

**Ejemplo de capa:** Una alarma domiciliaria

- **Hardware**: microcontrolador, sensor de movimiento, Sirena (PWM o GPIO).
- **Drivers**: código que abstrae el hardware (ej: driver GPIO, PWM, ADC).
- **Aplicación:** Lógica principal de la alarma:
  - Si un sensor detecta intrusión → activar buzzer y LED rojo
  - Si no hay intrusión → mantener LED verde encendido
  - Posibilidad de armar/desarmar la alarma con un botón




*Two-layer embedded-software architecture*




*Three-layer embedded-software architecture*

# ¿Preguntas o dudas?

